Funktionsapproximation: Interpolation & Splines*

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 27.08.2026

Opener-Quiz

Opener-Quiz

Beim Reinkommen: QR-Code scannen und die 2 Einstiegsfragen beantworten.

Frage 1:

Sei $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d$ eine *Basis* eines Vektorraums V. Welche Aussagen sind wahr?

1. Jedes $\boldsymbol{v} \in V$ hat eine **eindeutige** Darstellung $\boldsymbol{v} = \sum_{j=1}^{d} \alpha_j \boldsymbol{v}_j$.
2. Basisvektoren dürfen linear abhängig sein, solange sie V aufspannen.
3. Die Anzahl d der Basisvektoren ist die *Dimension* von V.

Frage 2:

Die Menge *aller* Funktionen $f\colon [a, b] \to \mathbb{R}$ bildet mit punktweiser Addition und Skalarmultiplikation einen Vektorraum. Welche Dimension hat er?

1. 2 (Intervallgrenzen a und b)
2. n (Anzahl der Datenpunkte)
3. endlich, aber sehr groß
4. Er ist unendlich-dimensional.

Verwendete Vorkenntnisse

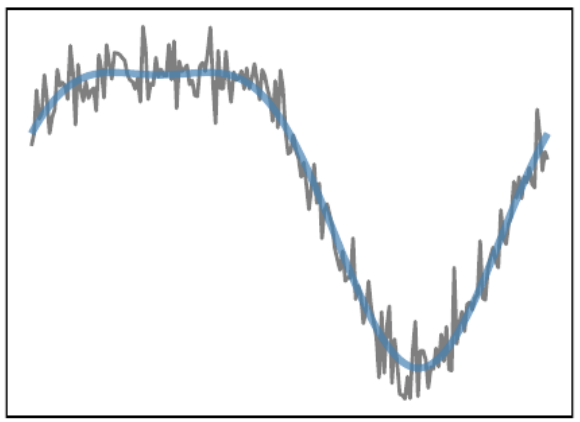
- **Lineare Algebra:**
  - *Lineare Gleichungssysteme* $\boldsymbol{Ba} = \boldsymbol{y}$ und deren Lösbarkeit
  - *Konditionszahl* einer Matrix als Maß für numerische Stabilität
  - *Basen* und *lineare Unabhängigkeit* (Basisdarstellung von Funktionen)
- **Analysis:**
  - Polynome und ihre Ableitungen (Polynomgrad, Monomialbasis)
  - Stetigkeit und Differenzierbarkeit
  - Stückweise definierte Funktionen
  - *Normen* (z.B. $\left\| \cdot \right\|_\infty$ für Konvergenzaussagen)

Einführung

Einführung

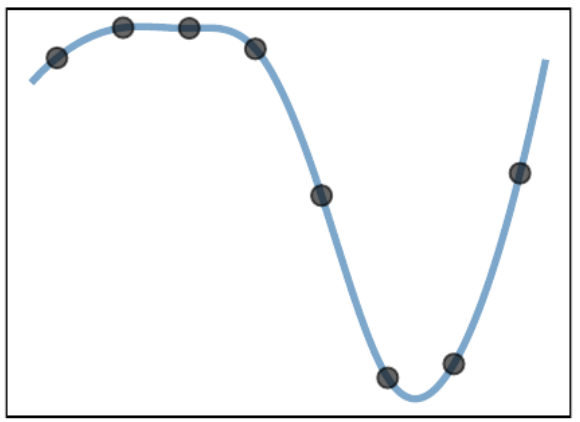
Drei Varianten: Finde $\widehat{f}$, so dass ...

Approximation



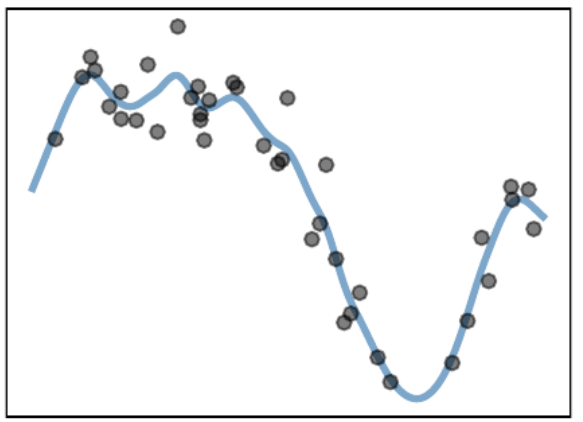
$$\|f - \widehat{f}\| \approx 0$$

Interpolation



$$f(x_i) = \widehat{f}(x_i) \quad \forall i$$

Glättung



$$y_i = \widehat{f}(x_i) + \varepsilon_i \quad \forall i$$

Bei der Interpolation sind die Daten *rauschfrei*, d.h. $y_i = f(x_i)$ – deshalb steht dort später auch $y_i = \widehat{f}(x_i)$.

Frage:
Wie sind die drei Varianten miteinander verwandt?

Kommentare

- Das *Approximationsproblem* lösen wir meist, indem wir es in ein *Interpolations*- oder *Glättungsproblem* überführen.
- Glättung ist die “schwierigste” Variante, weil die Fehler ε_i meist stochastisch sind.
- Wir kümmern uns zunächst um **Interpolation**.
- Wichtige Anwendungen:
  - Glatte Kurven zeichnen.
  - f approximativ, aber schneller auswerten.
    - **Beispiel:** $f(x) = \int_0^x g(y)dy$ oder $f(x) = g^{-1}(x)$ müssen numerisch bestimmt werden.
  - Ableitungen oder Integrale von f approximieren.

Anwendungen in ML und Data Science

- **Computergrafik:**
  Interpolation von Pixelwerten bei Reformatierung/Rotation von Bildern.
- **Zeitreihen:**
  Auffüllen fehlender Werte in Sensor- oder Finanzdaten.
- **Generative Modelle:**
  Latent Space-Interpolation in VAEs/GANs
- **NLP:**
  Positional Encodings in Transformern (“RoPE”) basieren auf Sinus-/Cosinus-Funktionen; *Long-Context*-Erweiterungen von RoPE interpolieren bzw. skalieren Positionsindizes.

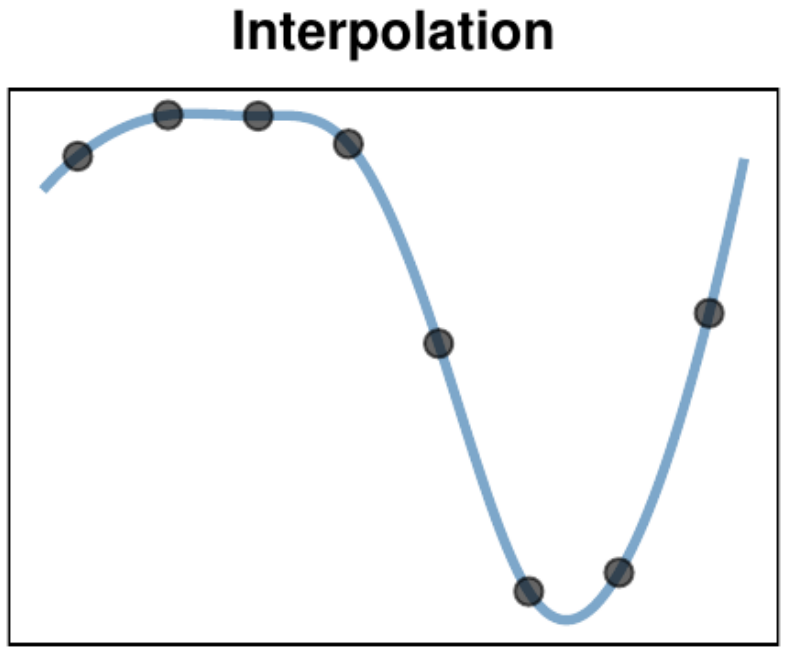
(teils Interpolation im engeren Sinn, teils verwandte Konzepte/Analogien)

Quiz: Interpolationsproblem

Problem:

Finde $\widehat{f}$, so dass

$$y_i = \widehat{f}(x_i), \quad i = 1, \ldots, n.$$



Welche der folgenden Aussagen ist wahr?

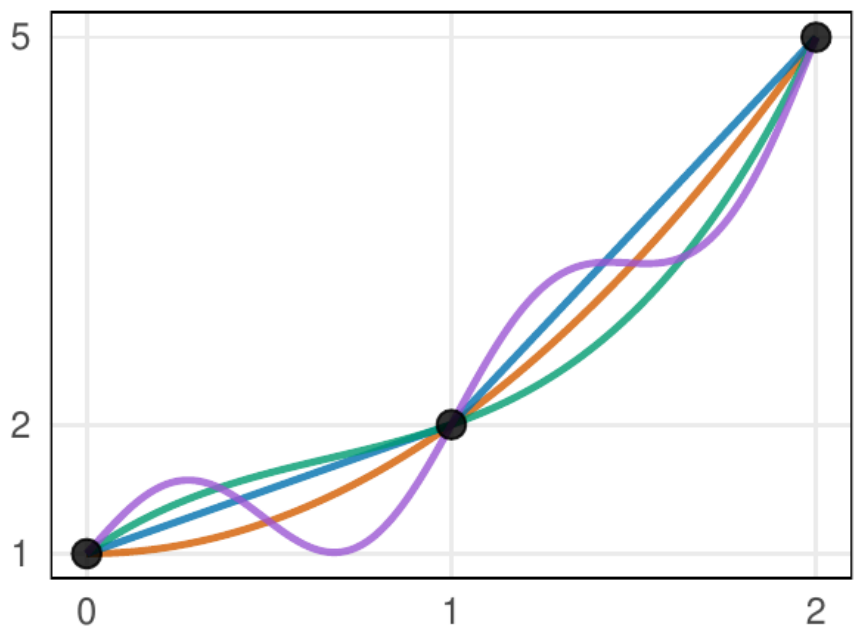
1. Es gibt unendlich viele Funktionen $\widehat{f}$, die das Problem lösen.
2. Aus $y_i = \widehat{f}(x_i), i = 1, \ldots, n$, folgt $\|f - \widehat{f}\| \approx 0$.
3. Das Problem hat eine eindeutige Lösung, wenn alle x_i verschieden sind.

Unendlich viele Lösungen: Beispiel

Betrachte $n = 3$ Punkte: $(0, 1), (1, 2), (2, 5)$.

Einige Interpolanten:

- **Parabel:** $\widehat{f}_1(x) = 1 + x^2$
- **Stückweise linear:** $\widehat{f}_2(x) = \begin{cases} x + 1 & x < 1 \\ 3x - 1 & x \geq 1 \end{cases}$
- **Kubisch:** $\widehat{f}_3(x) = x^3 - 2x^2 + 2x + 1$
- **Vogelwild:** $\widehat{f}_4(x) = 1 + x^2 + 0.5 \sin(2\pi x)$



Alle erfüllen $\widehat{f}(x_i) = y_i$, aber sehen völlig unterschiedlich aus!

Allgemein:

Jedes $\widehat{f} = \widehat{f}_1 + g$ mit $g(x_i) = 0$ für alle i ist ebenfalls ein Interpolant.

$\Longrightarrow$ Wir brauchen *Basissysteme* (Polynome, Splines), die die Lösungsmenge auf bestimmte Funktionenräume einschränken.

Funktionenräume und ihre Basen

Funktionenräume

- Menge aller Funktionen $f\colon [a,b] \to \mathbb{R}$ ist ein **Vektorraum**:
  - Addition: $(f+g)(x) = f(x) + g(x)$
  - Skalarmultiplikation: $(\alpha \cdot f)(x) = \alpha \cdot f(x)$
- Dieser Raum ist **unendlich-dimensional** $\implies$ $\exists$ unendlich viele Interpolanten für n gegebene Punkte!

Lösung:
Einschränkung auf endlichdimensionale Unterräume

Basen: von $\mathbb{R}^n$ zu Funktionenräumen

$\{\phi_1, \ldots, \phi_K\}$ ist eine **Basis** von $\mathcal{F}_K$ — *genauso* wie $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ eine Basis des $\mathbb{R}^n$ ist:

	Vektorraum $\mathbb{R}^n$	**Funktionenraum $\mathcal{F}_K$**
"Vektor"	$\boldsymbol{v} \in \mathbb{R}^n$	Funktion $f\colon [a, b] \to \mathbb{R}$
Basis	$\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$	$\phi_1, \ldots, \phi_K$
eindeutige Darstellung	$\boldsymbol{v} = \sum_{i=1}^{n} \alpha_i \boldsymbol{v}_i$	$f = \sum_{k=1}^{K} a_k \phi_k$
Koordinaten	$(\alpha_1, \ldots, \alpha_n)$	Koeffizienten $(a_1, \ldots, a_K)$
Dimension	n	K

- Alle Konzepte der linearen Algebra (lineare Unabhängigkeit, *Span*, Basiswechsel, …) übertragen sich direkt auf Funktionen — eine Funktion *ist* ein Vektor (in $\mathbb{R}^\infty$…)
- Statt eine Funktion (in $\mathbb{R}^\infty$) finden zu müssen, brauchen wir jetzt nur noch ihren **Koordinatenvektor $\boldsymbol{a} = (a_1, \ldots, a_K) \in \mathbb{R}^K$**
  $\to$ ein Problem der (numerischen) linearen Algebra!

Interpolation durch Basisdarstellung

Interpolation durch Basisdarstellung

- **Problem:**
  Finde $\widehat{f}$, so dass

$$y_i = \widehat{f}(x_i) \quad \forall i.$$

- **Lösung:**
  1. Nehme an, dass $\widehat{f}(x) = \sum_{k=1}^{K} a_k \phi_k(x)$ für
     - *Basisfunktionen* $\phi_1, \ldots, \phi_K$ (bekannt)
     - *Basiskoeffizienten* $a_1, \ldots, a_K$ (unbekannt)
  2. Löse das Gleichungssystem

$$y_i = \sum_{k=1}^{K} a_k \phi_k(x_i), \qquad i = 1, \ldots, n.$$

$\rightarrow$ *Interpolationsproblem* zurückgeführt auf *Lineares Gleichungssystem*!

Quiz: Vom Interpolationsproblem zum LGS

Das System

$$y_i = \sum_{k=1}^{K} a_k \phi_k(x_i), \qquad i = 1, \ldots, n$$

lässt sich als LGS $\boldsymbol{Ba} = \boldsymbol{y}$ darstellen. Was sind $\boldsymbol{B}, \boldsymbol{a}, \boldsymbol{y}$?

Beispiel: Basisdarstellung konkret

Gegeben:

$n = 3$ Datenpunkte $(0, 1), (1, 2), (2, 5)$, Monomialbasis $\phi_k(x) = x^{k-1}$, $K = 3$

$$\implies \boldsymbol{B} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad \boldsymbol{y} = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}$$

Lösung durch Einsetzen:

$$\begin{aligned} &\text{Zeile 1:} && a_1 = 1 && && \\ &\text{Zeile 2:} && a_2 + a_3 = 1 && \to a_2 && = 1 - a_3 \\ &\text{Zeile 3:} && 2a_2 + 4a_3 = 4 && \to 2 + 2a_3 && = 4 \end{aligned}$$

$$\implies a_3 = 1, a_2 = 0 \implies \widehat{f}(x) = 1 + x^2$$

Probe:

$\widehat{f}(0) = 1, \widehat{f}(1) = 2, \widehat{f}(2) = 5$ ✓

Polynominterpolation

Fundamentalsatz der Algebra

Satz: Fundamentalsatz der Algebra

Jedes Polynom

$$p(x) = a_n x^n + \ldots + a_1 x + a_0$$

mit $a_n \neq 0$ und $n \geq 1$ hat mindestens eine Nullstelle in $\mathbb{C}$.

Äquivalent:
Jedes Polynom vom Grad n hat genau n Nullstellen in $\mathbb{C}$ (mit Vielfachheit gezählt).

Folgerung:
Ein Polynom vom Grad $\leq n - 1$ mit n verschiedenen Nullstellen ist das Nullpolynom.

Beispiel:
Ein quadratisches Polynom (Grad 2) kann höchstens 2 (verschiedene) Nullstellen haben. Finden wir eines mit 3 Nullstellen, muss es $p(x) = 0$ für alle x sein.

Eindeutigkeit der Polynominterpolation

Satz: Polynominterpolation

Für n Paare $(x_i, y_i)_{i=1}^n$ mit paarweise verschiedenen x_i gibt es *genau* ein Polynom p vom Grad *höchstens* $(n-1)$ mit $p(x_i) = y_i$ für alle i.

Beweisskizze (Eindeutigkeit):

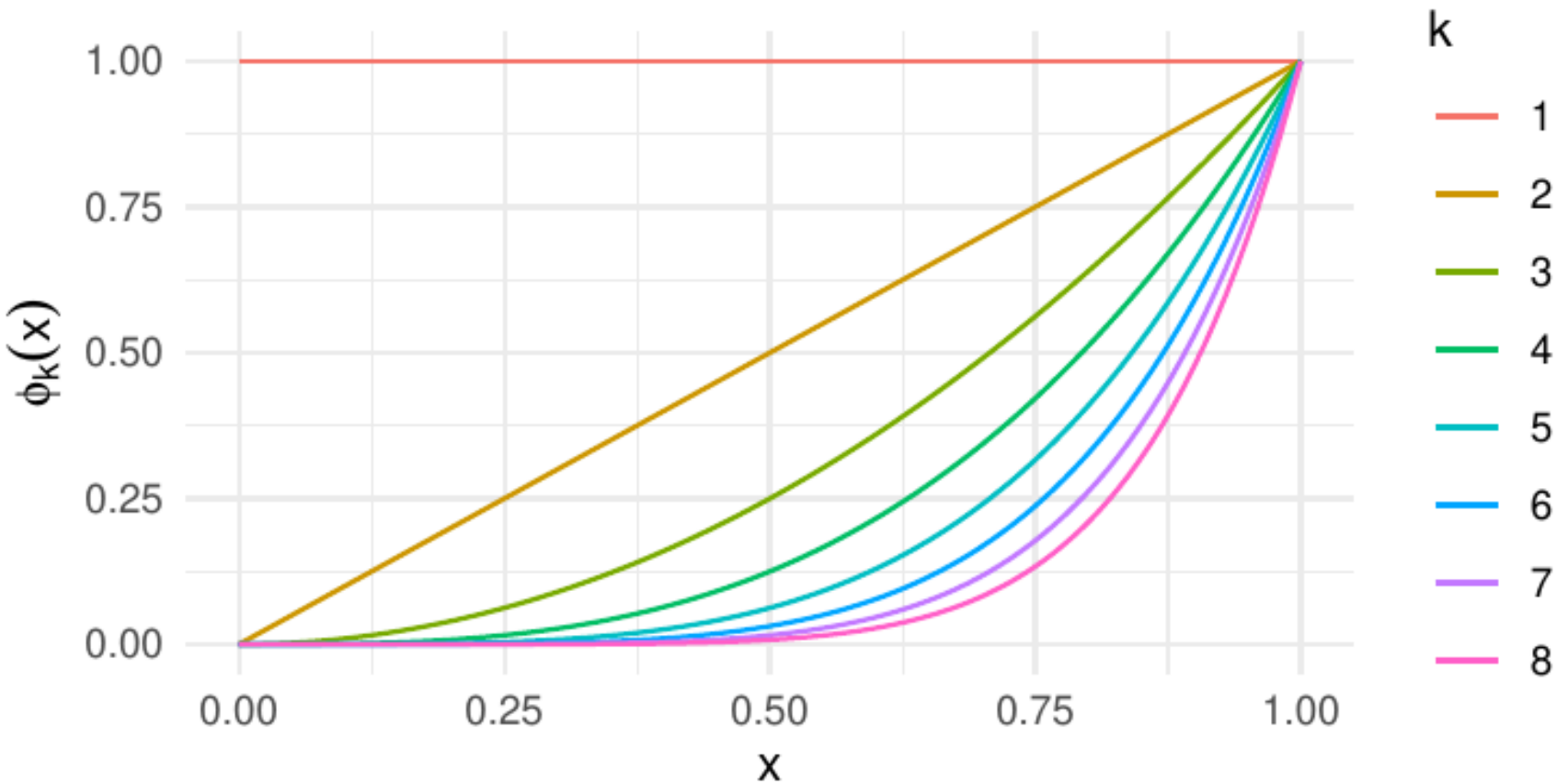
- Angenommen, p und q sind beides interpolierende Polynome vom Grad $\leq n-1$.
- Dann gilt: $(p-q)(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0$ für alle $i = 1, \ldots, n$.
- Also hat $p - q$ mindestens n Nullstellen, aber Grad $\leq n-1$.
- Nach dem *Fundamentalsatz*: $p - q$ muss das Nullpolynom sein.
- Daher $p = q$. ■

Polynominterpolation

- Alle Polynome vom Grad höchstens $(n-1)$ können durch die *Monomialbasis*

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \ldots, \quad \phi_n(x) = x^{n-1}$$

erzeugt werden.



Achtung: Häufiges Missverständnis

Es gibt **genau ein** Polynom von Grad **höchstens** $n-1$ durch n Punkte.

NICHT: Grad **genau** $n-1$!

Beispiel: $n=3$ Punkte $(0,0),(1,1),(2,2)$ liegen auf einer Geraden.

- Interpolierendes Polynom: $p(x)=x$ (Grad 1, nicht 2!)
- Das eindeutige Polynom *höchstens* 2. Grades ist: $p(x)=0\cdot x^2+1\cdot x+0$

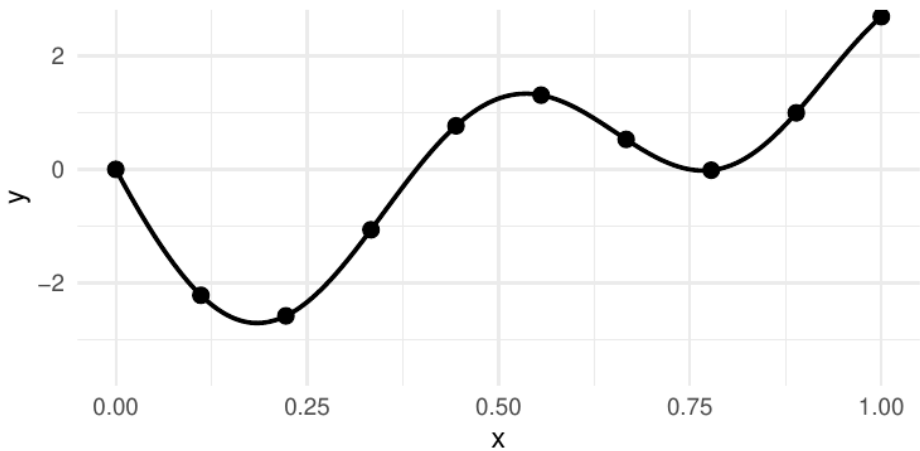
Polynominterpolation in R

```r
x <- seq(0, 1, length = 10)
y <- f(x)

# Vandermonde-Matrix: cbind(1, x, x^2, ...)
B <- outer(x, 0:9, "^")
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
fhat <- outer(xnew, 0:9, "^") %*% a

ggplot() +
  geom_point(aes(x, y)) +
  geom_line(aes(xnew, fhat))
```



Problem 1: Kondition

- *Monomialbasis* führt zu einer schlecht konditionierten Basismatrix $\boldsymbol{B}$:

$$\boldsymbol{x} = (x_1, \ldots, x_n)^\top \to \boldsymbol{B} = (1 \quad \boldsymbol{x} \quad \boldsymbol{x}^2 \quad \ldots \quad \boldsymbol{x}^{n-1})$$

  - Die Spalten sind die Vektoren $\boldsymbol{b}_k = (x_1^{k-1}, \ldots, x_n^{k-1})^\top$.
  - Für k groß sind diese Vektoren nahezu linear abhängig (vgl. Abbildung der Monomialbasis!)
    $\Longrightarrow$ Matrix nahezu singulär / schlecht *konditioniert*.

Numerisches Beispiel:
n äquidistante Punkte in $[0, 1]$, Polynom vom Grad $n - 1$:

n	**Polynomgrad**	$\kappa(\boldsymbol{B})$
5	4	$\approx 10^3$
10	9	$\approx 10^7$
15	14	$\approx 10^{12}$
20	19	$\approx 10^{16}$

(Größenordnungen für κ_2; andere Normen ergeben sehr ähnliche Werte.
Die Kondition hängt zudem von der Knotenlage ab: dieselben Punkte auf $[-1, 1]$ ergeben für $n = 20$ nur $\kappa_2 \approx 10^8$.)

Bei doppelter Präzision (≈ 16 Dezimalstellen) ist für $n \geq 20$ fast alle Genauigkeit verloren!

$\to$ verwende **orthogonalisierte** Basis (Legendre-, Chebyshev-, Hermite-Polynome) —
in `R`: **`cbind(1, poly(x, 3))`** statt `cbind(1, x, x^2, x^3)`

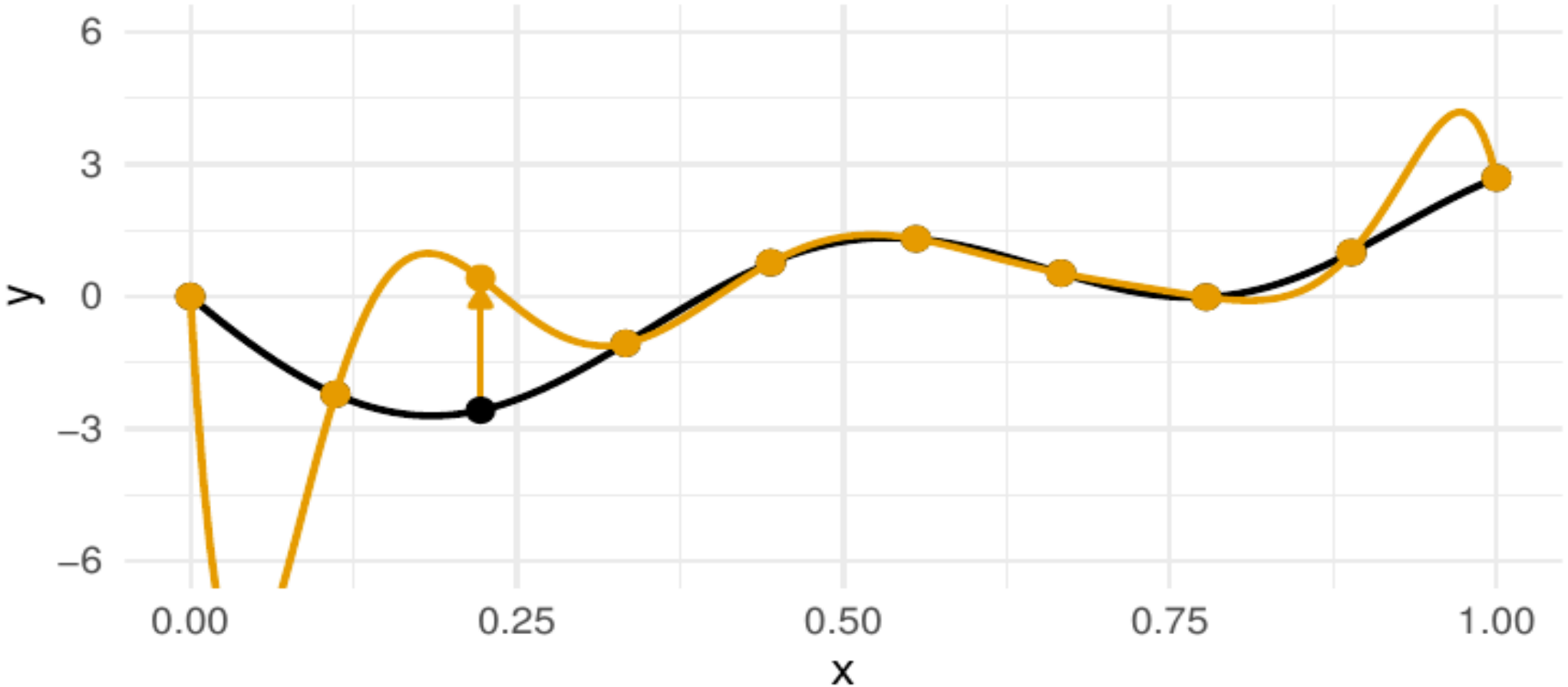
Orthogonale Polynome: Gram-Schmidt im Funktionenraum

- Auf $\mathcal{F}_K$ gibt es auch ein **Skalarprodukt**: $\langle f, g \rangle = \int f(x)\, g(x)\, dx$ (analog zu $\boldsymbol{x}^\top \boldsymbol{y}$ in $\mathbb{R}^n$)
- Damit funktioniert *Gram-Schmidt-Orthogonalisierung* ($\rightarrow$ *QR-Zerlegung*!) angewendet auf die Monome $\{1, x, x^2, \ldots\}$ über $[-1, 1]$:

$$p_2(x) = x^2 - \frac{\langle x^2, 1 \rangle}{\langle 1, 1 \rangle} \cdot 1 - \underbrace{\frac{\langle x^2, x \rangle}{\langle x, x \rangle}}_{=0} \cdot x = x^2 - \tfrac{1}{3} \propto \underbrace{\tfrac{1}{2}\left(3x^2 - 1\right)}_{\text{Legendre-Polynom } P_2}$$

- **Legendre-, Chebyshev-, Hermite-Polynome** = Gram-Schmidt bzgl. unterschiedlicher Skalarprodukte (Gewichtsfunktionen/Intervalle)
- orthogonale Basisfunktionen $\implies \boldsymbol{B}$ deutlich besser konditioniert.
- `poly(x, k)` in `R` orthogonalisiert bzgl. des *diskreten* (vektoriellen) Skalarprodukts $\sum_i f(x_i) g(x_i)$.

Problem 2: Stabilität und Lokalität



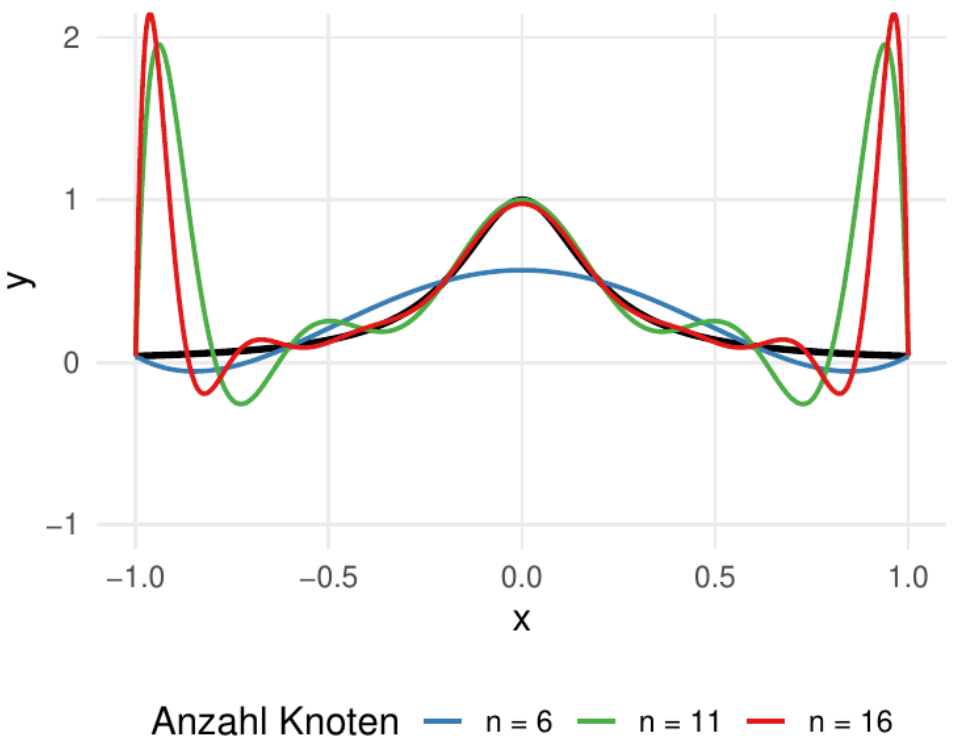
- Ändern wir nur ein y_i, verändert sich (potentiell) die gesamte Funktion. $\Longrightarrow$ Die Methode ist instabil.
- Dies ist ein grundsätzliches Problem von Polynombasen.

Das Runge-Phänomen

Runge-Phänomen (1901):

Interpoliere $f(x) = \frac{1}{1+25x^2}$ auf $[-1, 1]$ mit n äquidistanten Knoten durch p_{n-1} (Grad $n-1$).

- f ist glatt ($\in \mathcal{C}^\infty$!)
- Aber: $\|f - p_{n-1}\|_\infty \to \infty$ für $n \to \infty$!
- Das Wachstum ist **nicht monoton** (von $n = 5$ auf $n = 10$ fällt der Fehler zunächst); die Fehlermaxima wandern an den Rand.



Lösung:

Stückweise Polynome statt eines globalen Polynoms $\to$ **Splines**

Splines

Splines

Def.: Polynom-Spline

Sei $[a, b] \subset \mathbb{R}$ ein Intervall.

Ein **Polynom-Spline q-ten Grades** ist eine Funktion $s\colon [a, b] \to \mathbb{R}$,

- definiert auf einem *Gitter* von *Knoten*

$$a = \xi_0 < \xi_1 < \ldots < \xi_m = b,$$

- so dass

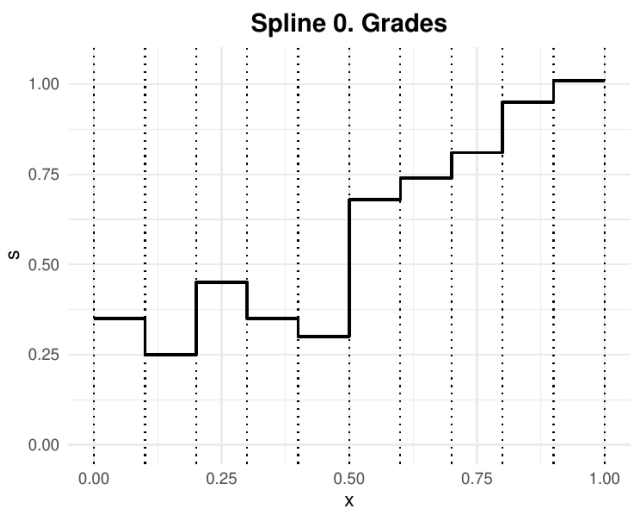
$$s(x) = p_k(x) \quad \text{für } x \in [\xi_{k-1}, \xi_k) \quad (\text{bzw. } x \in [\xi_{m-1}, \xi_m] \text{ für } k = m)$$

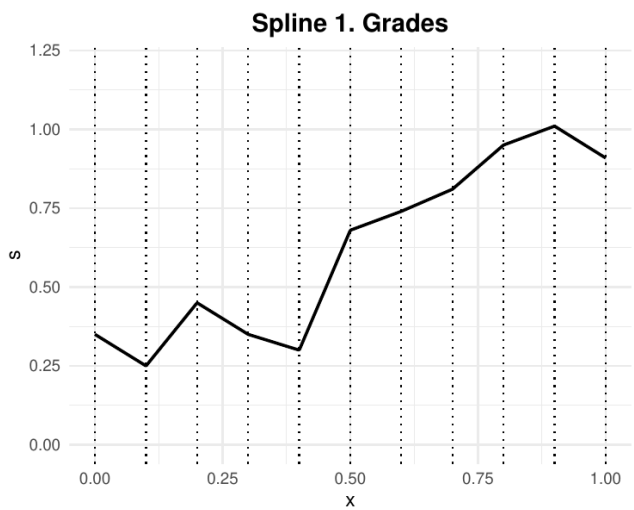
  mit *Polynomen p_k vom Grad höchstens q*, $k = 1, \ldots, m$,
- mit $(q-1)$ stetigen Ableitungen: $s \in \mathcal{C}^{q-1}$.

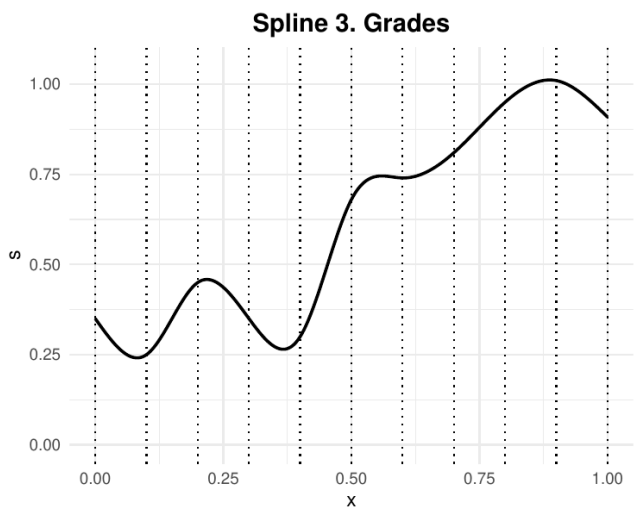
Splines

Weniger formal:

> Ein Spline ist eine Funktion, die Polynome an den Punkten $\xi_1, \ldots, \xi_{m-1}$ zusammensetzt.







Warum $m + q$ Parameter?

Annahme: einfache innere Knoten, maximale Glattheit C^{q-1}

Parameterzählung:

- Ohne Stetigkeitsbedingungen: $m(q+1)$ Parameter (m Intervalle, je $q+1$ Koeffizienten)
- An jedem der $(m-1)$ inneren Knoten: q Bedingungen (Stetigkeit von $s, s', \ldots, s^{(q-1)}$)
- $\Longrightarrow \; m(q+1) - (m-1) \cdot q = m + q$

Interpolation **an $n = m + 1$ Knoten** mit kubischem Spline ($q = 3$):

$m + 3$ Freiheitsgrade aber nur $n = m + 1$ Interpolationsbedingungen
$\Longrightarrow$ Es fehlen **2 Bedingungen**!

Gängige *Randbedingungen*:

Typ	**Bedingung**
Natürlich	$s''(a) = s''(b) = 0$ (lineare Enden)
Eingespannt	$s'(a), s'(b)$ vorgegeben
Periodisch	$s'(a) = s'(b),\ s''(a) = s''(b)$ (für $y_0 = y_m$)

Natürliche Splines sind der Standard, wenn keine Randinformation vorliegt.

Kubischer Spline: Konstruktion Schritt für Schritt

Gegeben:
4 Punkte $(0,0), (1,1), (2,0), (3,-1) \implies m = 3$ Intervalle

Ansatz:
Je Intervall ein kubisches Polynom $p_k(x)$ mit 4 Koeffizienten.

Bedingungen:

Typ	Anzahl	Beispiel
Interpolation	4	$p_1(\xi_0) = y_0,\ p_k(\xi_k) = y_k$
Stetigkeit	2	$p_1(1) = p_2(1)$
s' stetig	2	$p_1'(1) = p_2'(1)$
s'' stetig	2	$p_1''(1) = p_2''(1)$
Randbedingung (natürlich)	2	$s''(0) = s''(3) = 0$

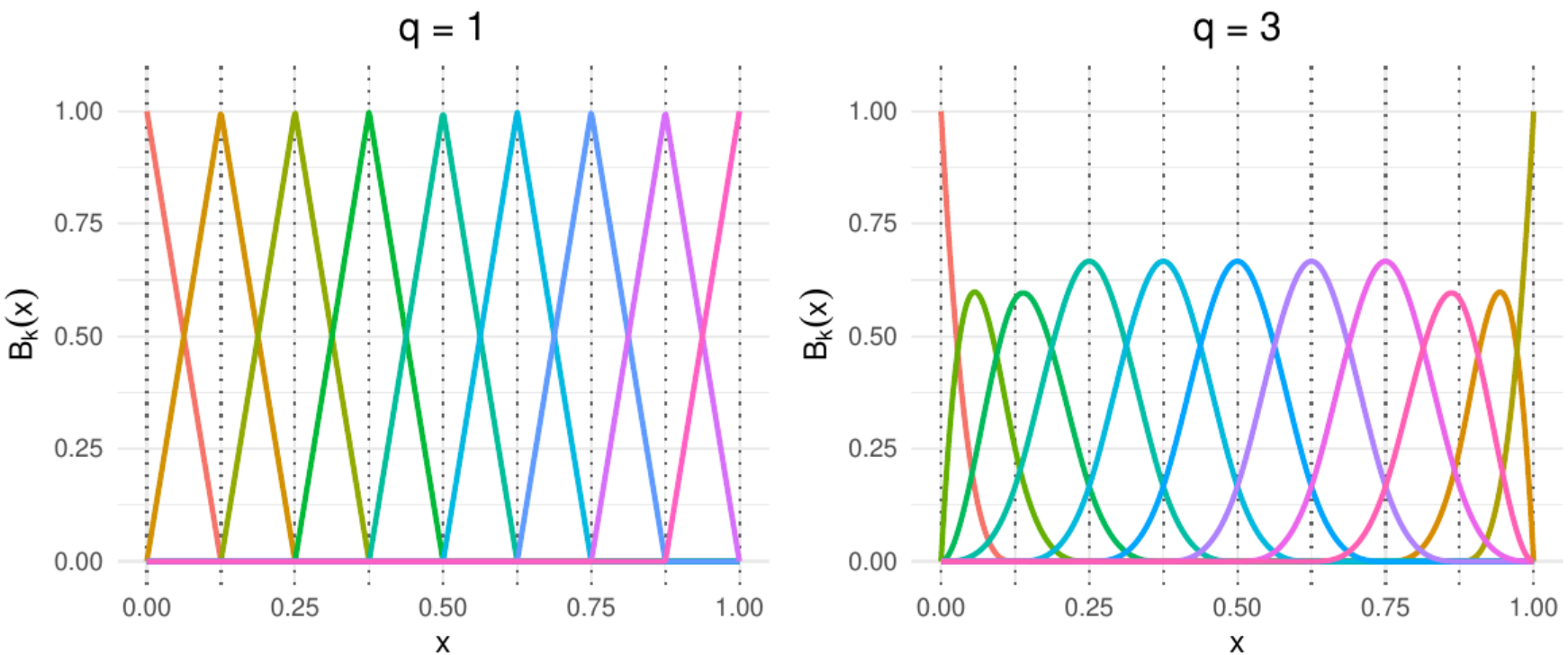
Summe: $4 + 2 + 2 + 2 + 2 = 12$ Bedingungen für $3 \times 4 = 12$ Unbekannte

$\implies$ LGS ist eindeutig lösbar!

B-Splines

- Jeder Spline von Grad q kann auch durch eine Linearkombination von $m + q$ **B-Spline**-Basisfunktionen B_k zum Grad q repräsentiert werden:

$$s(x) = \sum_{k=1}^{m+q} a_k B_k(x)$$



B-Splines: Rekursive Definition

- B-Splines können auf verschiedene Arten definiert werden.
- Rekursive Definition (*Cox-de-Boor-Rekursion*; nur zur Referenz/Transparenz):
  - Erweitere die Knotenfolge zu $\tau_1, \ldots, \tau_{m+2q+1}$:

$$\tau_1 = \cdots = \tau_{q+1} = \xi_0, \quad \tau_{q+1+i} = \xi_i \ \ (i = 1, \ldots, m-1), \quad \tau_{m+q+1} = \cdots = \tau_{m+2q+1} = \xi$$

  - $q = 0$:

$$B_k^{(0)}(x) = \mathbb{1}(\tau_k \leq x < \tau_{k+1})$$

  - $q > 0$:

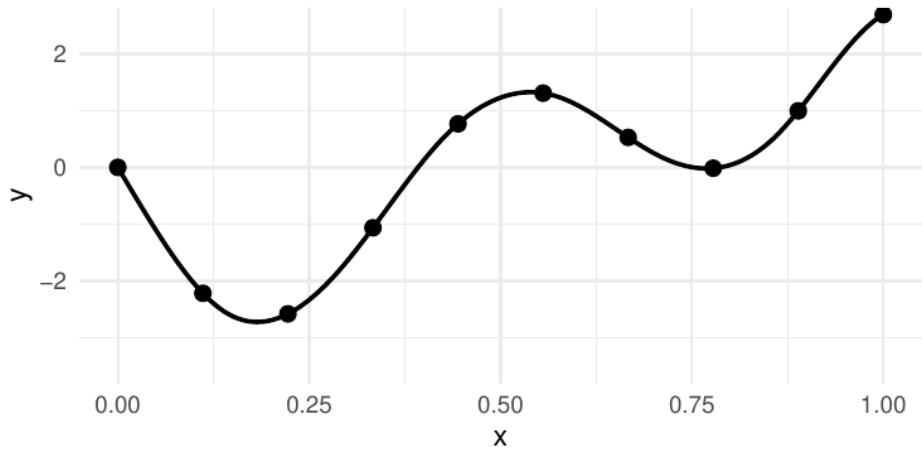
$$B_k^{(q)}(x) = \frac{x - \tau_k}{\tau_{k+q} - \tau_k} B_k^{(q-1)}(x) + \frac{\tau_{k+q+1} - x}{\tau_{k+q+1} - \tau_{k+1}} B_{k+1}^{(q-1)}(x)$$

- in R: `splines::bs()`

B-Splines in R

```r
y <- f(x)
B <- splines::bs(x, df = 10, degree = 3,
                 intercept = TRUE)
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
Bnew <- splines::bs(xnew,
                    knots = attr(B, "knots"),
                    intercept = TRUE)
fhat <- Bnew %*% a
```



B-Splines: Numerische Stabilität & Effizienz

Vergleich der Konditionszahlen für $n = 20$ Punkte in $[0, 1]$:

Basis	Polynomgrad	Konditionszahl
Monomialbasis	19	$\kappa\left(\boldsymbol{B}_{\text{mono}}\right) \approx 10^{16}$
B-Splines (kubisch, $K = 20$ Basisfunktionen)	3	$\kappa\left(\boldsymbol{B}_{\text{spline}}\right) \approx 10^{2}$

B-Splines sind **rund 14 Größenordnungen** besser konditioniert!

(Größenordnungen, hier κ_2 mit den Standardknoten von `splines::bs()`: $1{,}2 \cdot 10^{16}$ vs. 35; die genauen Werte hängen von Norm und Knotenwahl ab.)

Außerdem: Dünn besetzte *Bandstruktur* (nur $q + 1$ Einträge pro Zeile) der B-Spline-Matrix ermöglicht

- LGS-Lösung in $O(nq^2)$ statt $O(n^3)$
- Für $n = 1000$, $q = 3$: Beschleunigungsfaktor $\approx 100\,000$

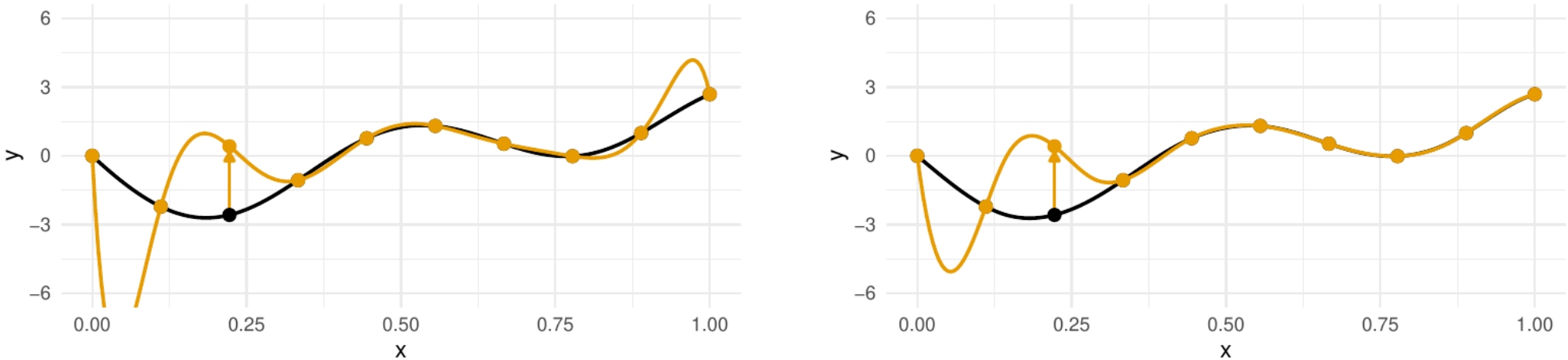
$\Longrightarrow$ B-Splines sind der Standard in der Praxis, trotz komplizierter Definition.

↩ zurück: „(B-)Splines: Eigenschaften"

(B-)Splines: Eigenschaften

- Splines sind das gängigste Basissystem zur Funktionsapproximation, meist **kubisch** ($q = 3$):

1. Wenn $\max_k |\xi_k - \xi_{k-1}|$ hinreichend klein ist, kann jede stetige Funktion f beliebig genau approximiert werden.
2. Die Ableitung eines Grad-q-Splines ist ein Grad-$(q-1)$-Spline, das Integral ein Grad-$(q+1)$-Spline.
3. Ableitungen/Integrale eines Splines approximieren auch die Ableitungen/Integrale der Zielfunktion — für die Ableitungen allerdings nur, wenn f selbst entsprechend oft differenzierbar ist, und mit je einer Ordnung langsamerer Konvergenz.
4. B-Splines sind numerisch stabil (vgl. B-Splines: Numerische Stabilität & Effizienz);
   haben *lokale Träger*: $B_k(x) = 0$ für $x \notin [\tau_k, \tau_{k+q+1}] \implies$ Approximation ist **lokal**, also stabiler;
   $\boldsymbol{B}$ hat *Band-Struktur* $\implies$ effizient lösbar.



Polynominterpolation (links) vs. Spline-Interpolation (rechts): Änderung *eines* Datenpunkts.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- *Approximation, Interpolation, Glättung*
- *Funktionenräume*: Funktionen als Vektoren, *Basisfunktionen* wie Basisvektoren
- Lösungsansatz durch *Basisdarstellung*: Interpolation $\rightarrow$ LGS $\boldsymbol{Ba} = \boldsymbol{y}$
- Polynombasen und ihre Probleme (Kondition, *Runge-Phänomen*); Ausweg: orthogonale Polynome = Gram-Schmidt im Funktionenraum
- *Splines, B-Splines* und ihre Vorteile (Lokalität, Stabilität, Effizienz)

Nächste Vorlesung: Funktionsapproximation II — Glättungsproblem, Bias-Varianz-Trade-off

Self-Check

Frage 1: Zur Charakterisierung eines Splines q-ten Grades auf $m+1$ Knoten benötigen wir...

1. $q+1$ Parameter.
2. $m(q+1)$ Parameter.
3. $m+q$ Parameter.

Frage 2: Ein kubischer Spline auf dem Knotengitter $\xi_0 < \xi_1 < \cdots < \xi_5$ (6 Knoten). Wie viele Basiskoeffizienten hat er?

1. 6
2. 8
3. 9
4. 24

Frage 3: Warum verwendet man in der Praxis B-Splines statt der Monomialbasis? *(mehrere Antworten möglich)*

1. Die B-Spline-Basismatrix ist deutlich besser konditioniert.
2. Nur B-Splines können jede stetige Funktion beliebig genau approximieren.
3. Lokale Träger $\rightarrow$ Bandstruktur $\rightarrow$ effiziente Lösung des LGS.
4. Mit der Monomialbasis hat das Interpolationsproblem keine eindeutige Lösung.

Ausblick: Weiterführende Themen

Weiterführende Themen & Vorlesungen:

- **Mehrdimensionale Interpolation:** Tensor-Produkt-Splines, radiale Basisfunktionen
- **Glättung mit Regularisierung:** P-Splines, Generalisierte Additive Modelle ($\rightarrow$ *Statistik V: Konzepte Statistischer Modellierung*)
- **Moderne ML-Anwendungen:** Spline-basierte Aktivierungsfunktionen (*KANs*), *Normalizing Flows*